TECNOLOGIAS
E ARQUITETURA

How Requirement Gathering influences Technical Debt and Quality in Software Development

Master's in Computer Science and Business Management

Gonçalo Gonçalves Miranda

Supervisor(s):
PhD Maria Cabral Diogo Pinto Albuquerque
Assistant Professor, ISCTE-IUL
PhD Luísa Cristina da Graça Pardal Domingues Miranda
Assistant Professor, ISCTE-IUL

ISCTE – Instituto Universitário de Lisboa
June, 2025

Acknowledgement

Write here the acknowledgments and grants, if any

Abstract

In software engineering, the quality of Requirements Engineering (RE) is a key factor when predicting long-term success and sustainability of software systems. Requirements are the basis for any decision in the project, wether it is related to design, implementation, or maintenance. However, in practice, requirements are often incomplete, ambiguous, or unstable. These conditions provide a breeding ground to what is known as Requirements-Related Technical Debt (RTD). RTD captures the trade-offs made during the requirements phase that lead to architectural, functional, or documentation failures, which lead to increased maintenance costs, reduced adaptability, and long-term quality degradation.

While the general concept of Technical Debt (TD) has been thoroughly studied, the influence of requirement-related factors remains underdeveloped in the particular context of big corporations, where AGILE and fast-paced development environments are predominant and where decisions made under pressure or with little stakeholder involvement can silently build up as debt, with consequences that may only be detected down the line. Despite existing recommendations to neutralize such problems, the industry still suffers to consistently identify, track, and prevent RTD before it negatively impacts software evolution.

In order to tackle this gap, this research combines a Systematic Literature Review (SLR) of 30 publications with a case study elaborated in cooperation with a software company. The purpose is to consolidates existing work on RTD causes and mitigation approaches, and to validate these findings through empirical investigation. By doing this, the research aims to yield practical insights and theoretical clarity to support more resilient and sustainable RE practices.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

As modern software systems increase in complexity and scale, the difficulty of maintaining stakeholders expectations aligned with system behavior also becomes more difficult and consequential, while also being more critical than ever (Venters et al., 2023). Alongside that, both the quality and sustainability of a system's architectures heavily rely on early development lifecycle activities, with a huge focus on the quality of the requirement engineering (RE) process (Rios et al., 2019). According to Behutiye et al. (2022), the robustness of this process must match the system's complexity, in order to ensure the system remains maintainable, adaptable and aligned with the long-term vision of the quality goals of the project. According both to Behutiye et al. (2022) and Melo et al. (2022), this means the rigor, clarity, and completeness of the captured requirements influence not only the short-term development process but also the system's ability to progressively adapt to different contexts and new demands. Failing to attentively perform the activities related to this early phase of software engineering increases the likelihood of accumulated architectural and design flaws, paves the way for technical debt (TD).

Although awareness of the risk incomplete, unstable or ambiguous requirements pose to a project is rising, many continue to suffer due to this factor, particularly in fast-paced environments where adaptability are prioritized over documentation and long-term planning, such as contexts related to AGILE and startups (Behutiye et al., 2022; Gupta et al., 2020). Though AGILE methods may be beneficial for swift delivery and user-centric development, they often tend to undermine non-functional requirements (NFRs) and a solid architectural design, which contributes to challenges in the long run, such as the need to refactor certain parts of the system or even quality degradation (Snoeck & Wautelet, 2022), being the dynamic between short-term delivery goals and long-term software robustness becomes evident in cases where requirements deviate halfway through development or are under-specified.

Research in requirements-related technical debt (RTD) - which is one of the most common types of TD, according to Wattanakriengkrai et al. (2018) - unveils that decisions made under uncertainty or pressure tend to output "quick fixes" that accumulate over time and hinder future changes, testing, or scale, as stated by Melo et al. (2022). In theory, these trade-offs are not inherently harmful, as they can be useful tools when correctly managed. However, in practice, RTD is usually left unidentified or undocumented during the requirements phase, resulting in developer teams that are oblivious to the existence of such issues, until they cripple development progress, as documentation regarding quality requirements (QRs) is usually less of a priority for development teams, regardless of the potential problems this is likely to cause in the long-term, according to Behutiye et al. (2022).

The risk introduced by poor RE is particularly alarming in organizations that rely on swiftly composed or distributed teams. Due to this, this study aims to explore the influence of requirement gathering practices on the accumulation of TD and the quality of software outputs, while exploring what are the most common causes of RTD and how they can be identified and correctly mitigated in a real world environment.

CHAPTER 2

Related Work

2.1. Systematic Literature Review

In order to summarize all current relevant academic knowledge regarding the topic subject to research, to identify research gaps and to indicate new areas for posterior investigation, this study was conducted as a Systematic Literature Review (SLR) and followed the guidelines defined by Kitchenham, Charters, et al. (2007), that divides the research into 3 different stages: the review planning, the review conduction, and the review report. As per the authors, a systematic approach is predefined using a review protocol that creates research questions and the methods used in the review, in order to target such questions. This allows for the study to be repeatable, which improves its transparency and enables any reader to assess a study's rigor and integrity (Kitchenham, Charters, et al., 2007).

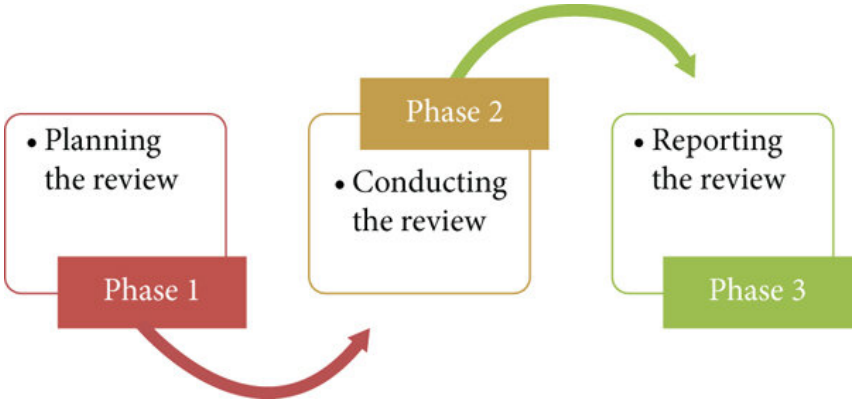


Figure 1. SLR Phases

2.1.1. Review Planning Stage

This section refers to the SLR methodology's first stage, where the rationale for conducting the review should be outlined, as well as the research questions, and where the review protocol is defined.

2.1.1.1. Review Motivation

Despite growing awareness of TD in software engineering, the impact of RE on its mitigation and accumulation remains underexplored. Unreliable requirements are often mentions as key actors to RTD, mainly in environments where documentation and QRs are often de-prioritized, such as AGILE and fast-paced development environments (Behutiye et al., 2022). Melo et al. (2022) states that, while causes and metrics for RTD have been already draft by some studies,

current work lacks a systematic and comprehensive compilation of how these issues are generated, as well as their impact on long-lasting product quality, and how they can be addressed correctly. Case studies that concern both small and large projects reveal that poor requirement gathering techniques can lead to extensive refactoring due to product misalignment with client expectations and to increased maintenance costs, indicators that are highly correlated with TD (Gupta et al., 2020). This SLR is produced based on the need to clarify the connection between deficient RE and TD, to understand their long-term effects on software quality (SQ), and to identify ways to predict and mitigate RTD in heterogeneous development contexts.

2.1.1.2. Research Questions

The focus of this review is to address the following questions:

RQ.1: How do incomplete or evolving requirements contribute to the accumulation of TD in software development projects?

RQ.2: What is the relationship between the quality of requirement gathering practices and software product quality outcomes over time?

RQ.3: Which requirement-related factors most commonly lead to TD, and how are they identified and mitigated in practice?

RQ.4: What RE process is most commonly linked to TD, and how can they improve to mitigate it?

RQ.5: How do different project contexts and team structures affect the emergence or management of RTD?

RQ.6: How is the quality of business and system requirements related to technical debt?

How do different project contexts and team structures affect the emergence or management of RTD?

2.1.1.3. Review Protocol

The databases considered for the search are SCOPUS[1], the ACM Digital Library[2], and the IEEE Xplore Digital Library[3] .

2.1.2. Review Conducting Stage

This section represents the second stage in the SLR methodology, where the protocol's application is described. The analysis of the extracted data also occurs during this stage.

[1]https://www.scopus.com/
[2]https://dl.acm.org/
[3]https://ieeexplore.ieee.org/Xplore/home.jsp

2.1.2.1. Research Identification

Starting from this study's research question, the necessary keywords to perform the research were extracted in order to generate an insightful search string, as shown in **Table 1**.

Keywords	Technical Debt, Requirements Technical Debt, Requirement Debt, Requi
Search String	*("Technical Debt" OR "Requirements Technical Debt" OR "Requirement Debt") AND ("Re*

Table 1. SLR Keywords and Search string

2.1.2.2. Study Selection

For this study, 7 filters where used to thin down the search result, listed on **Table 2**.

ID	Filter
1	Title, Abstract or Keywords
2	2018-2025
3	English
4	Articles, Conference Papers, Short Papers, Reviews and Conference Reviews
5	15 or more Citations
6	Manual
7	Backward Snowballing Search

Table 2. Search Filters

The research process was commenced with an initial, filter-less search of the selected search string across the 3 selected databases, which provided an output of 4408 results. Then, the first filter was introduced, ”Title, Abstract or Keywords,” which enabled a reduction in the number of studies to 295, followed by the application of the filter ”2018-2025,” that reduced the number of studies to be considered to 203. Afterwards, the third filter, ”English”, was applied, subtracting 5 more studies from the current pool, bringing it down to 198. After this, the ”Articles, Conference Papers, Short Papers, Reviews and Conference Reviews” filter was applied, thinning down the possible studies to 151. However, the biggest decrease in number of articles was introduced by the fifth filter, ”15 or more Citations”, that downsized the study pool to only 29. Subsequently, a manual reading of all the studies allowed to both filter duplicate results and to select only relevant studies. With this, there was a reduction of 5 more studies, bringing the number of selected studies down to 25. Finally, this reading also enabled the application of the seventh filter: the ”Backward Snowballing Search”, which was based on the relevance of the studies and their citations. Only studies that met prior filters and acceptance criteria were selected, resulting in a definitive pool size of 30 articles, which were considered the most relevant studies, with this research’s goal in mind. Besides this, 2 articles were selected to introduce both the SLR and the research methodology in use. **Table 3** and **Figure 2**, synthesize the process of the selection of the studies.

Database	No Filter	Filter 1	Filter 2	Filter 3	Filter 4	Filter 5	Filter 6	Filter 7
SCOPUS	1765	182	132	127	126	22	19	4
IEEE Xplore Digital	1705	86	53	53	8	4	3	0
ACM Digital Library	938	27	18	18	17	3	3	1
Total	4408	295	203	198	151	29	25	5

Table 3. SLR Filtration Process

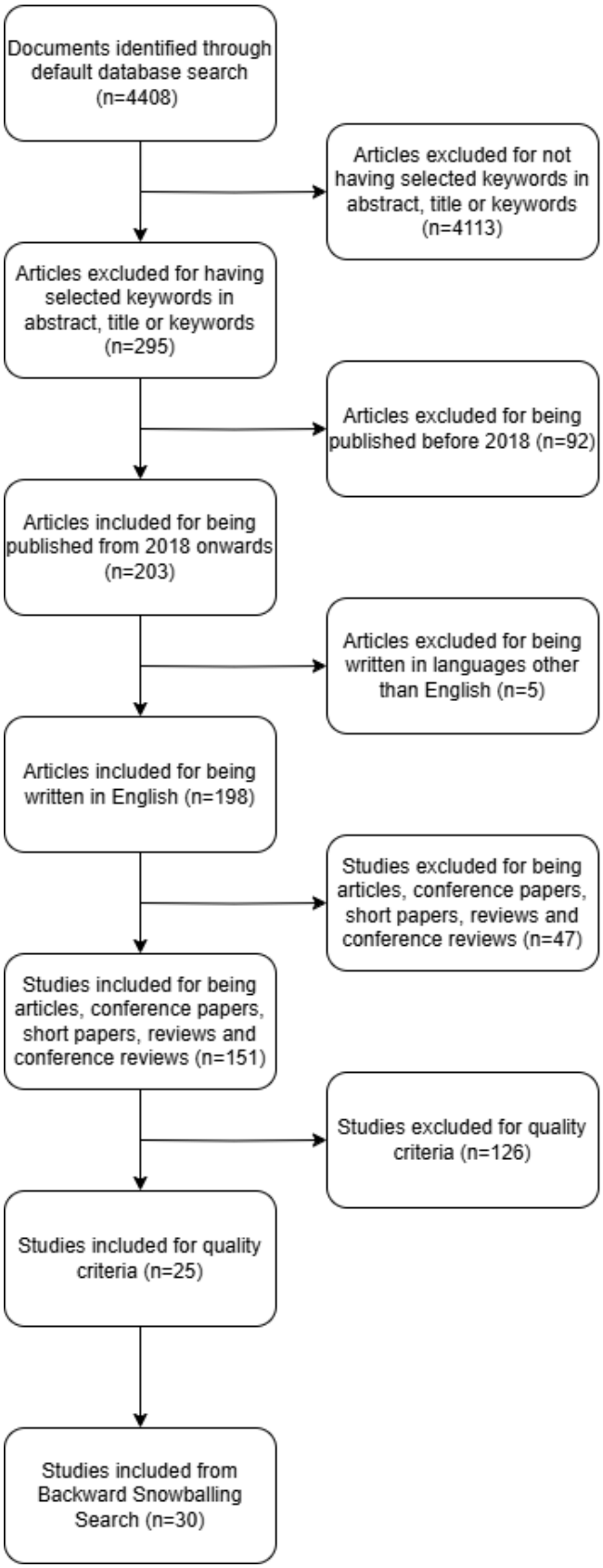
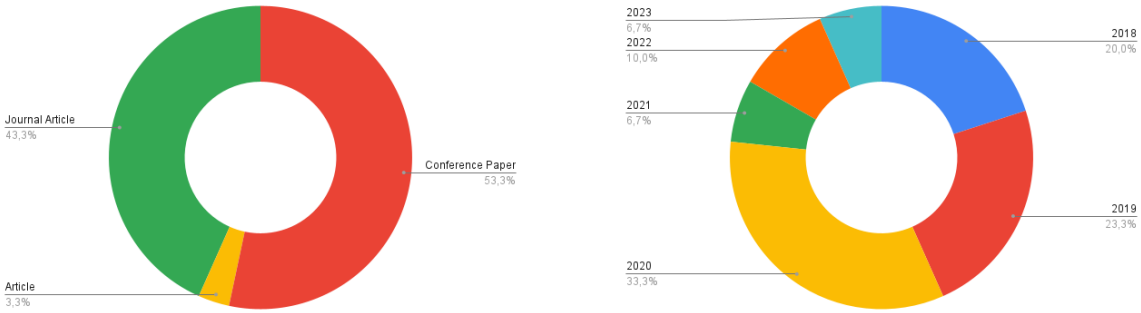


Figure 2. Flow of the Filtration Process

2.1.2.3. Data Extraction and Analysis

Across the 30 studies selected, most were conference papers (53.3%) or journal articles (43.3%), while only 3.3% of the selected studies were generic articles, as shown in **Figure 3**. As for the distribution of the studies by publication year, the most predominant year taken into consideration was 2020, with a significance of 33.3%, followed by 2019 (23.3%) and 2018 (20.0%). Together, these three publication years have an importance of more than 75%, as only 23.4% of the analyzed studies were published between the years of 2021 and 2023. This may be the case, as the awareness of AGILE and rapid development practices limitations increases, mainly regarding the management of long-term SQ and the disregard of NFRs (Behutiye et al., 2020; Melo et al., 2022).



(a) Distribution of Studies by Content Type

(b) Distribution of Studies by Year

Figure 3. Statistics

2.1.3. Review Reporting Stage

This section regards the final phase of the SLR methodology. It encapsulates the extracted data and answers to the outlined research questions.

2.1.3.1. Data Summary

To improve the current understanding on the influence of multiple factors of requirements gathering in TD and SQ, a collection of key terms utilized in the literature to describe a multitude of concepts regarding RTD was determined across the 30 studies. **Table 4** summarizes all the main key term, as well as the number of studies in which each one is used, and the corresponding reference identifiers to those usages. This analysis aims to emphasize both the relevance of certain concepts and the diversity of perspectives and terminologies described across multiple studies.

Table 4. References by Key Terms

Key Term	**References by ID**	**Total**
Requirements Engineering	S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S13, S14, S15, S16, S17, S18, S19, S20, S21, S22, S23, S24, S25, S26, S27, S28, S29	29
Technical Debt	S2, S3, S4, S5, S6, S8, S9, S10, S11, S12, S13, S14, S15, S16, S17, S18, S19, S20, S21, S22, S23, S24, S25, S26, S27, S28, S29, S30	28
Evolving Requirements	S23, S27	2
Incomplete Requirements	S8, S17, S24	3
Quality Requirements	S1, S3, S7, S8, S9, S13, S14, S16, S17, S23, S26, S27, S28, S29, S30	15
AGILE	S3, S4, S5, S6, S7, S8, S13, S15, S16, S17, S18, S19, S21, S22, S23, S24, S25, S26, S27, S28	20
Startups	S3, S5, S15, S18, S19, S24, S27	7
Requirement Documentation	S17, S19, S24, S28	4
Non-Functional Requirements	S1, S3, S4, S5, S6, S7, S8, S9, S10, S11, S13, S14, S16, S17, S18, S19, S20, S21, S23, S24, S25, S26, S27, S28, S30	25
Self-Admitted Technical Debt	S1, S2, S9, S11, S12, S14, S17, S19, S20, S24, S30	11
Maintainability	S3, S4, S5, S6, S7, S8, S9, S10, S13, S14, S15, S16, S17, S18, S20, S21, S22, S23, S24, S25, S26, S27, S28, S29, S30	25
Modifiability	S23, S26, S29	3
Scalability	S7, S8, S10, S11, S16, S19, S23, S25, S26, S27, S30	11
Refactoring	S1, S3, S4, S6, S8, S9, S10, S13, S14, S15, S16, S17, S19, S20, S21, S22, S23, S24, S25, S26, S27, S29, S30	23
Preventive Actions	S4, S6, S15, S16, S17, S30	6
Causes Of Technical Debt	S4, S5, S6, S17, S25, S30	6
Mitigation Strategies	S2, S4, S5, S6, S9, S13, S14, S15, S17, S18, S20, S22, S23, S24, S25, S28, S30	17
Requirements Technical Debt	S6, S9, S12, S13, S14, S17, S20, S24, S25, S30	10

However, it was also possible to identify several recurring themes regarding the role of requirements gathering and its connection to TD and SQ. These were then grouped into four main topics: the Relation of RE with SQ and TD, the Requirement Volatility and Incompleteness as Debt Factors, the AGILE, Startups, and Fast-Paced Development Environments, and the Documentation, Traceability, and NFRs. Table5 displays both the number of studies that address each one of the previously mentioned topics and their respective references.

Table 5. References for Each Main Topic Identified

Main Topic	**References by ID**	**Total**
Relation of Requirement Engineering with Software Quality and Technical Debt	S1, S2, S3, S4, S6, S8, S13, S14, S17, S24, S26, S28, S30	13
Requirement Volatility and Incompleteness as Debt Factors	S8, S17, S23, S24, S27	5
AGILE, Startups, and Fast-Paced Development Environments	S3, S4, S5, S6, S15, S18, S19, S24, S25, S27, S28	11
Documentation, Traceability, and Non-Functional Requirements	S1, S3, S7, S9, S10, S11, S13, S16, S17, S24, S28, S30	12

Finally, there was another common ground between the studies in use: the challenges faced by development teams when managing TD through requirement practices. Five of the main challenges discussed across current literature are: Capturing and Documenting NFRs, Managing Requirements Volatility and Change, Ensuring Alignment Between Stakeholders and Evolving System Goals, Preventing Long-Term Architectural Degradation through RE Practices, and Refactoring and TD Mitigation Strategies. **Table 6** outlines the amount of studies in which each of the aforementioned challenges are cited and provides the respective reference identifiers.

Table 6. Challenges Identified in the Literature

Challenge	**References by ID**	**Total**
Capturing and Documenting Non-Functional Requirements	S1, S3, S6, S7, S8, S9, S10, S14, S17, S18, S24, S28, S30	13
Managing Requirements Volatility and Change	S6, S8, S17, S23, S24, S27	6
Ensuring Alignment Between Stakeholders and Evolving System Goals	S2, S4, S6, S13, S16, S19, S21, S23, S27	9
Preventing Long-Term Architectural Degradation through RE Practices	S3, S4, S5, S13, S14, S15, S17, S25, S26, S30	10
Refactoring and Technical Debt Mitigation Strategies	S1, S4, S6, S9, S13, S17, S20, S22, S23, S25, S28, S30	12

These three tables provided a structured overview of themes explored by current literature. As the data extraction phase aims to provide a deeper understanding of how RE practices relate to the genesis and mitigation of TD, the mapping of these dimensions plays a crucial role into providing these insights.

2.1.3.2. Findings Report

RQ.1: How do incomplete or evolving requirements contribute to the accumulation of TD in software development projects?

Literature consistently identifies incomplete or mutable requirements as foundational causes of TD, especially RTD (Melo et al., 2022; Waltersdorfer et al., 2020). Throughout the evaluation of current work, it is clear that the end product of projects based on vague, unstable, or weakly-elaborated requirements is often over complicated and flawed at the architecture level.

Both Rios et al. (2020) and Seyff et al. (2018) suggest that incomplete requirements often tend to induce developers to make assumptions that bypass formal analysis and validation, resulting in fragile implementations that, in the long-term, create the need to perform code refactoring. Teams resorting to faultier design patterns when NFRs like performance, security, or usability are not explicitly mentioned in the documentation is an example of this, as it may lead to scalability or maintainability issues later on, as mentioned by Farias et al. (2020) and Wattanakriengkrai et al. (2018), studies in which it is visible how such gaps amplify the likelihood of Self-Admitted Technical Debt (SATD) - that usually takes form of either comments or workaround code.

AGILE and fast-paced development environments are the main subjects of this issue. As illustrated by Gupta et al. (2020), Villamizar et al. (2018), and Vogelsang (2020), changing requirements usually emerge iteratively, and occasionally without sufficient alignment between technical teams. This leads to a snowball effect, where architectural and design choices are reconsidered without adequate planning, incentivizing a reactive development culture, as these constant shifts disrupt the stability of the system's foundations and architecture, which, as the complexity increases, become more complex to refactor.

The solution's security risk is also increased by the absence of traceability (Rindell & Holvitie, 2019; Rindell et al., 2019), as in critical fields such as security, vulnerabilities and long-term maintenance liabilities may be introduced due to misaligned or undocumented requirements shifts. Additionally, Melo et al. (2022) highlights that the deficiency of formalized procedures for handling mutable requirements leads to suboptimal prioritization, which then results in codebase embedded trade-offs.

As mentioned by Behutiye et al. (2022), changing requirements require appropriate TD management strategies, as the absence of such approach is likely to compromise system and design integrity. All things considered, the reviewed studies all point to the fact that incomplete and evolving requirements are the perfect conditions for the genesis of RTD, that can only be minimized by adopting correct approaches to the problem, such as robust change management, comprehensive NFRs documentation, and regular requirement validation cycles.

RQ.2: What is the relationship between the quality of requirement gathering practices and software product quality outcomes over time?

Current literature highlights a strong and evident connection between good quality requirement gathering practices and increased software product quality over time (Femmer & Vogelsang, 2019). It is suggested that the deployment of methodical, structured RE practices tend to increase a system's robustness, maintainability, and improves coherence with stakeholder expectations.

Behutiye et al. (2022) argues that even in settings where documentation is often minimal, such as AGILE, integrating methodic requirement practices enhances the compliance between delivered features and long-term goals regarding the system architecture, as the amount of misunderstandings and defects can be significantly decreased by applying methods such as the maintenance of lightweight yet complete requirement documentation, stakeholder verification, and version tracking. However, this is not the only author that defends this perspective. In fact, Melo et al. (2022) goes as far as to support it with evidences, showing that in-depth and well-validated requirements enhances both cross-team communication and the on-boarding of new team members, while also generating records regarding the decision-making process, which all lead to a decreased number of misaligned implementations, which tend to lead to RTD.

A connection can also be noted between well-defined requirements and system usability, security, and stability. Recurrent redesigns and regression bugs are usually the result of badly specified requirements - especially NFRs - as these are in assuring long-term system performance (Femmer & Vogelsang, 2019). This is also corroborated by Venters et al. (2023), that suggests that system sustainability, which is a metric that covers changeability, performance, and longevity, is a direct consequence of good RE practices.

Siddiqi et al. (2020) and Vogelsang (2020) also emphasize the importance of requirement quality, as they found that low-quality RE processes forces teams to develop reactively, meaning that issues are only resolved real-world failures or constraints, which increases maintenance costs and also degrades stakeholder trust and decreases the likelihood of further investment.

Moreover, organizations who deploy strong RE practices are more predisposed to embrace proactive TD management strategies (Waltersdorfer et al., 2020). These organizations also enforce architectural reviews linked to requirement analysis and maintain traceability records linking requirements, tests, and deliveries, leading to a decreased ambiguity and allowing teams to be more effective at refactoring.

At last, current work indicates that RE quality is a deeply tied to SQ. By utilizing proper documentation, stakeholder involvement, traceability, and iterative validation, requirement gathering practices lead development towards more consistent, scalable, and maintainable solutions, while increasing their quality.

RQ.3: Which requirement-related factors most commonly lead to TD, and how are they identified and mitigated in practice?

RTD manifests as an ongoing, multi-dimensional challenge in the literature, largely caused by a composition between structural weaknesses during the RE process and organizational practices that lack emphasis on long-term maintainability (Melo et al., 2022). While it is true that

RTD may be dependent on multiple variables, that can independently or collectively increase it, current work systematically emphasizes a pattern in the underlying causes, along with practical methods to both identify and mitigate RTD.

One of the main causes of RTD is the lack of proper specification and documentation of NFRs. Teams usually tend to primarily focus functional aspects due to time-critical deadlines, thus overlooking critical quality aspects such as product performance, security, and scalability (Behutiye et al., 2022; Farias et al., 2020), which has consequences, as the system evolution continues without structural architecture planning required to fulfill these expectations, meaning that deliveries end up architecturally fragile and employing quick fixes that reflect accumulated debt rather than intentional design.

An aggravating factor to this problematic is the absence of traceability and useful documentation, as mentioned by Rios et al. (2020) and Wattanakriengkrai et al. (2019). According to the authors, the decentralization of requirement sources, varying from informal meetings to dispersed documentation tools, is a pivotal element in decreasing consistency, as teams cannot confidently asses the impact certain implementation might have when changes or the reasoning behind certain decisions are not recorded. This leads to the build up of uncertainty, that subsequently encourages design decisions that emphasize short-term results rather than architectural stability.

The nonalignment between business stakeholders and development teams also leads to the accumulation of RTD; current work shows that the failure to establish communication between the aforementioned actors tends to result in divergent understandings of what a requirement includes. Business needs are prone to swiftly evolving or shift priorities, but without a shared vision, empowered by structured debates or shared artifacts, development teams can utilize outdated or unfinished requirements, resulting in misaligned implementations that are noticeable after their execution, when the cost of refactoring is increasingly bigger than the cost of correctly implementing the correct approach from the get go (Gralha et al., 2018; Seyff et al., 2018).

Another regularly verified problem is the pace at which requirements progress without adequate architectural revision. Although iterative and AGILE methodologies embrace change, current work alerts for the risk that constant iteration, without well-planned refactoring, poses, as it inputs systemic misalignment between the initial design and its implementation. This is demonstrated by Rindell et al. (2019) in respect to security-related features, and how not updating threat models or re-validating architectural assumptions when trying to adapt to evolving business needs compromised projects. Similarly, Villamizar et al. (2018) proposes that requirement volatility requires the presence of thorough change management, in order to prevent the dissemination of underlying debt across related modules.

The deviation between RE practices and subsequent technical processes also contributes to the accumulation of RTD. When requirement specification documents are not seamlessly unified with design, testing, and deployment streams, an increase on deficiencies is noticed, yet, they are not flagged until late stages of the development lifecycle. According to Venters

et al. (2023), the absence of alignment previously mentioned obstructs the organization from a holistic perspective on system quality, constraining their capacity to preemptively address problems.

Current literature suggests a range of mitigation and identification strategies regarding these challenges. Examples of this are the use of automated tools to detect SATD within codebases by analyzing comments and commit messages that may display signs of requirement deviations (Li et al., 2020) or the deployment of frameworks dedicated to assess the requirement health, such as those by mentioned by Lenarduzzi and Fucci (2019), which present metrics to evaluate the completeness, clarity, and stability of requirement artifacts. Another possible strategy to identify the described challenges is the implementation of bidirectional traceability systems, that enable early the detection of requirement violations and better impact analysis upon change introduction (Siddiqi et al., 2020).

Integrating mitigation approaches in development team culture and workflows is beneficial, as performing frequent stakeholder meetings, recursive refinement of requirement documents, and the inclusion of RE reviews in sprint retrospectives all lead to less RTD build-up. Rios et al. (2020) suggests that the integration of RE practices into AGILE ceremonies, such as backlog refinements and sprint reviews, helps on the early recognition and remediation of requirements-related issues that impact product quality. Finally, existing research supports the utilization of explicit debt trackers, in which identified RTD occurrences must be documented, prioritized, and prepared for rectification (Melo et al., 2022).

In conclusion, requirement-related variables that contribute to TD are highly correlated with both technical approaches and organizational culture. The incidence and severity of RTD can be decreases by fostering requirement clarity, traceability, and integrating RE with technical processes, and criticality of RTD. Melo et al. (2022) emphasizes that ambiguous, poorly specified, untraceable requirement tend to make software deliveries more prone to refactoring and architecture degradation The author also mentions that well-established practices for the governance leads teams to face less long-term quality issues.

RQ.4: What RE process is most commonly linked to TD, and how can they improve to mitigate it?

Across the analysed literature, the accumulation of TD is commonly associated with AGILE and fast-paced environments, largely due to the pressure associated with these environments and with how requirements are handled under pressure. Behutiye et al. (2020) highlights that QRs are usually deprioritized over functional requirements (FRs), which leads to the underdocumentation of QRs, while also states that TD in AGILE Software Development is often generated by inadequate or missing knowledge of QRs and FRs.

Besides this, due to its focus being continuos delivery, AGILE Software Development is more prone to TD accumulation than traditional software development (Rios et al., 2019).

When tackling the situation from a processual standpoint, the recurring option is to strengthen RE in AGILE environments with concrete practices and artifacts that revolve focus on the solution's architecture and QRs, while still not compromising AGILE's characteristic iterative nature when creating deliverables, as Snoeck and Wautelet (2022) argue that TD is often caused by the lack of architectural thinking associated with AGILE, and propose an integrated method that enables a decrease of TD by engineering the architecture in a logical, consistent and complete manner.

RE processes can be improved by highlighting TD and RTD and by improving the workflow to introduce governance practices, in order to avoid reactive prioritization, as Besker et al. (2019) report that TD prioritization is commonly reactively and lacks the usage of specific strategies and that TD items are often managed and prioritized in an alternative backlog. In conclusion, current literature consistently correlates AGILE/fast-paced contexts to TD through QRs neglect, as well as frail documentation and reactive handling of TD, while the improvements mentioned are the reinforcement of AGILE RE practices with explicit QRs and architecture practices, as well as the integration of the backlogs in which TD is tracked into the main backlog of the development teams.

RQ.5: How do different project contexts and team structures affect the emergence or management of RTD?

RTD's existence is strongly dependant on the project's technical and social context, especially on the organizational constraints, stakeholder fragmentation, and on the degree of cross-team awareness over dependencies. In complex, industrial settings, the automotive development is, according to Vogelsang (2020), extremely influenced by legacy systems and organizational constraints, as well as OEM/supplier relationships, which worsens the coordination and generates challenges regarding RE. Vogelsang (2020) also notes that these contexts are favorable for RTD-relevant dependencies to remain imperceptible, reinforcing that a team's structure and the way knowledge is distributed are directly related to how early requirement-linked issues are detected. In multi-stakeholder environments with various artifacts, the validation and verification of exchanged artifacts becomes, according to Waltersdorfer et al. (2020), "cumbersome and error-prone". This, of course, increases the risk of mismatches between requirements and real implementation.

However, RTD management is not just a technical problem - there is also a interpersonal and organizational dimention to it - which Melo et al. (2022) conclude is not properly addressed. Overall, the literature converges on the idea that larger or more heterogeneous projects, with legacy constraints, multiple stakeholders, and fragmented tooling, are more prone to RTD and are harder to proactively manage. Besides this, having teams with limited awareness of dependencies and weak cross-stakeholder validation also increase the likelyhood of late discovery and costly remediation of RTD.

2.1.4. SLR Conclusions

This SLR investigated the link between RE and TD, particularly focusing on the causes and of effects of RTD. Throughout all 30 analyzed studies, it becomes evident that the quality and management of requirements are key factors when analyzing the success and quality of a software system.

A key insight that can be extracted by this study is that incomplete or ever-changing requirements have a very meaningful impact on the compounding of TD. Regardless of the development philosophy, wether it is AGILE or a more traditional development context, the failure to solidly identify or iteratively validate requirement is strongly linked to the development of fragile architectures, heavy refactoring costs and to the need for long-term maintenance. The research also implies that reactive development practices, in which there is no update to neither the documentation nor to the architecture when accommodating unforeseen changes, are especially harmful, as they build-up over time and blur the original goals of the system.

Another key factor to mention is the impact that requirement gathering quality has in it comes to the quality of software deliverables. Requirement practices that involve stakeholders, thorough documentation, and traceability, are linked to higher-quality software, as solutions are more likely to be more maintainable, scalable, and user-aligned. Contrarily, informal or fragmented RE processes are heavily linked to miscommunication between developer and business teams, misalignment between deliverables and stakeholder expectations, and improvised development decisions, which are a breeding ground for RTD. Current work supports the argument that investing in good practices of RE early on results in lasting advantages in SQ and cost-efficiency.

This study also unveils requirement-related variables that increase TD: it stresses the risk of overlooking NFRs, undocumented changes, and communication gaps between business and technical standpoints. Both cultural and technical shifts are necessary to address these issues. Processes such as SATD detection, requirement traceability, the involvement of stakeholders in the elaboration of requirement artifacts, and the upkeep of explicit TD repositories represent concrete methods teams can use to abandon reactive TD management in favor of a more proactive approach.

Collectively, the outcomes of this review have both academic value and real world applications. As for professionals, the study aims to unequivocally position RE as more than just a preliminary phase of development. This process must be everlasting and evolve alongside the system. For the academy, the review points out research gaps, such as the limited focus on proactive RTD prevention and the influence of organizational culture on RTD build-up. NFRs and domain-specific contexts, such as safety-critical systems, for instance, are also underdeveloped. Future research should focus on developing and deployed prevention frameworks, researching team and organizational variables, and improving NFRs management to reduce long-term cost and respective debt.

In conclusion, this review reinforces that TD is a controllable outcome of intentional design decisions, rather than an inevitable effect of fast-paced development. By treating requirements

with the necessary rigor and strategic attention they require, development teams are enabled to create systems that are not only functional, but also sustainable.

2.2. Multivocal Literature Review

In addition to the SLR, a Multivocal Literature Review (MLR) was also conducted, complementing the findings previously obtained from academic literature with grey literature sources, so that more practical perspectives can also be taken into consideration, as the software engineering industry is in constant evolution, and new practices, tools and frameworks are frequently adopted by teams in this industry (Garousi et al., 2019). The MLR aims to capture insights that may not still be presented in academic publications (and, therefore, that might have not been captured by the previous SLR), thus providing a more well-rounded view of the relationship between RE and TD and what are the practices utilized in the real world to handle this subject. Another goal of this MLR was to also provide better comprehension on how the industry currently defines TD and how requirements engineering processes have been adapted over time to better tackle TD management. In order to conduct the MLR, the search string was tweaked, so that it matches the exact goal of this literature review - represented on **Table 7**. Pieces of grey literature were also filtered by selection criteria previously defined, displayed in **Table 8**.

Search String	””Requirement” AND ”Technical” AND ”Debt” in ”Software””

Table 7. MLR Search string

ID	Filter
1	Must be relevant for the study research
2	English
3	Source must be trusted

Table 8. Search Filters

CHAPTER 3

Research Methodology

This study follows a mixed approach based in Design Science Research (DSR), which, according to Brocke et al. (2020), is a "a problem-solving paradigm that seeks to enhance human knowledge via the creation of innovative artifacts". However, as the study strives to output both an artifact while also an explanation for a real-world phenomenon, it also integrates Case Study Research (CSR), which can be used "before the design of the artifact... to evaluate some preliminary forms or some parts of the artifact" (Costa & A. L. Soares, 2016).

3.1. Design Science Research

DSR sets the foundation this dissertation. As described by (Brocke et al., 2020), DSR is "a problem-solving paradigm that seeks to enhance human knowledge via the creation of innovative artifacts", which has as its main goal to "extend the boundaries of human and organizational capabilities by designing new and innovative artifacts" (Brocke et al., 2020).

In order to apply this methodology, the problem must be identified, as well as solution objectives, which will lead to the design and development of the final artifact, which will then be demonstrated, evaluated, and communicated (Brocke et al., 2020), displayed in **Figure 4**.

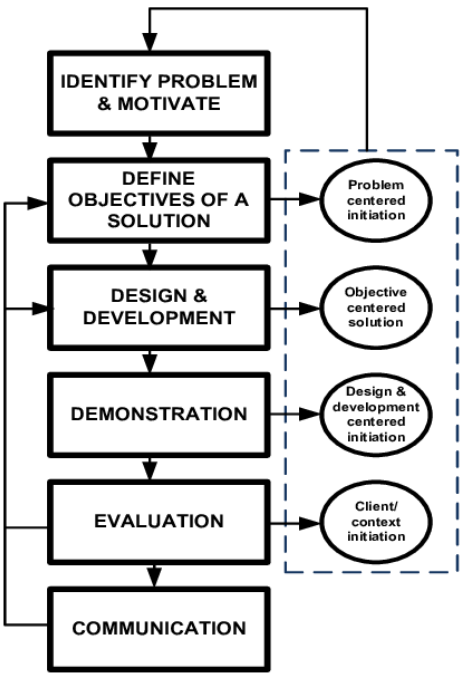


Figure 4. Design Science Research Phases

3.2. Case Study

The study also implements a case study methodology, outlined by Runeson and Höst (2009), to explore the impact of RE practices on RTD in a corporate context. The purpose of this case study is to identify and scrutinize how real-world software development teams within the studied company approaches RE and RTD in daily workflows.

The study will collect data by means of interviews, document analysis, and monitoring of project artifacts and sprint ceremonies. Insights will be triangulated in order to comprehend how TD accumulation is impacted by requirement quality, documentation, and traceability. The findings will be evaluated relative to the outcomes of the SLR to validate, contrast, and extend established knowledge. In conclusion, the study seeks to contribute to the generation of relevant, customized suggestions for improving RE practices in the company.

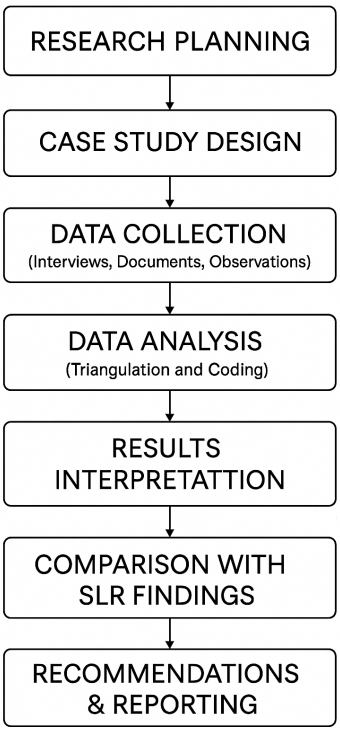


Figure 5. Case Study Elaboration Phases

3.3. Hybrid Methodology

The methodological design of this dissertation combines DSR and CSR. This mixed approach is adopted in order to both analyse an empirical phenomenon and develop an artifact, as Costa and A. L. Soares (2016) emphasizes that CSR provides essential "knowledge about the empirical context", enabling a better and more precise definition of the problem, stakeholders, and requirements for the artifact in development.

This supports *ex-ante* activities, such as refining objectives and grounding design decisions in real organizational practices.

Nabukenya (2012) reiterates the value of utilized an hybrid methodology, arguing that "combining CSR, DSR and AR methods is most suitable to support the execution" of research closely tied organizational practices.

Together, the merger of these two methodologies creates a coherent research strategy that promotes precision, contextual relevance, practical utility and ensures that the most current industry practices are being taken into consideration.

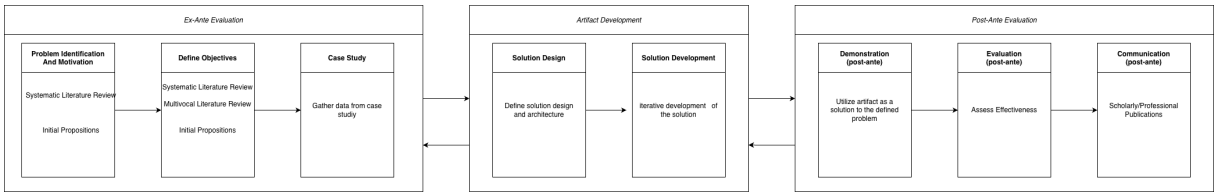


Figure 6. Research Methodology Phases

References

Behutiye, W., Karhapää, P., López, L., Burgués, X., Martínez-Fernández, S., Vollmer, A., Rodríguez, P., Franch, X., & Oivo, M. (2020). Management of quality requirements in agile and rapid software development: A systematic mapping study. *Information and Software Technology*, *123*. https://doi.org/10.1016/j.infsof.2019.106225

Behutiye, W., Rodríguez, P., Oivo, M., Aaramaa, S., Partanen, J., & Abhervé, A. (2022). Towards optimal quality requirement documentation in agile software development: A multiple case study. *Journal of Systems and Software*, *183*. https://doi.org/10.1016/j.jss.2021.111112

Besker, T., Martini, A., & Bosch, J. (2019). Technical debt triage in backlog management. *Proceedings 2019 IEEE ACM International Conference on Technical Debt Techdebt 2019*, 13–22. https://doi.org/10.1109/TechDebt.2019.00010

Brocke, J. v., Hevner, A., & Maedche, A. (2020, November). Introduction to design science research. https://doi.org/10.1007/978-3-030-46781-4_1

Costa, E., & A. L. Soares, a. J. P. d. S. (2016). Situating case studies within the design science research paradigm: An instantiation for collaborative networks. In A. Hamideh, L. M. Camarinha-Matos, & L. S. António (Eds.), *Collaboration in a hyperconnected world* (pp. 531–544). Springer International Publishing. https://doi.org/https://doi.org/10.1007/978-3-319-45390-3_45

Farias, M., Neto, M., Kalinowski, M., & Spínola, R. (2020). Identifying self-admitted technical debt through code comment analysis with a contextualized vocabulary. *Information and Software Technology*, *121*. https://doi.org/10.1016/j.infsof.2020.106270

Femmer, H., & Vogelsang, A. (2019). Requirements quality is quality in use. *IEEE Software*, *36*, 83–91. https://doi.org/10.1109/MS.2018.110161823

Freire, S., Rios, N., Mendonça, M., Falessi, D., Seaman, C., Izurieta, C., & Spínola, R. O. (2020). Actions and impediments for technical debt prevention: Results from a global family of industrial surveys. *Proceedings of the 35th Annual ACM Symposium on Applied Computing*, 1548–1555. https://doi.org/10.1145/3341105.3373912

Garousi, V., Felderer, M., & Mäntylä, M. V. (2019). Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology*, *106*, 101–121. https://doi.org/https://doi.org/10.1016/j.infsof.2018.09.006

Gralha, C., Damian, D., Wasserman, A. I. (, Goulão, M., & Araújo, J. (2018). The evolution of requirements practices in software startups. *Proceedings of the 40th International*

Conference on Software Engineering, 823–833. https://doi.org/10.1145/3180155.3180158

Gupta, V., Fernandez-Crehuet, J., & Hanne, T. (2020). Fostering continuous value proposition innovation through freelancer involvement in software startups: Insights from multiple case studies. *Sustainability Switzerland*, *12*, 1–35. https://doi.org/10.3390/su12218922

Kitchenham, B., Charters, S., et al. (2007). Guidelines for performing systematic literature reviews in software engineering.

Lenarduzzi, V., & Fucci, D. (2019). Towards a holistic definition of requirements debt. *International Symposium on Empirical Software Engineering and Measurement*, *2019-Septe*. https://doi.org/10.1109/ESEM.2019.8870159

Li, Y., Soliman, M., & Avgeriou, P. (2020). Identification and remediation of self-admitted technical debt in issue trackers. *Proceedings 46th Euromicro Conference on Software Engineering and Advanced Applications Seaa 2020*, 495–503. https://doi.org/10.1109/SEAA51224.2020.00083

Li, Y., Soliman, M., & Avgeriou, P. (2023). Automatic identification of self-admitted technical debt from four different sources. *Empirical Software Engineering*, *28*. https://doi.org/10.1007/s10664-023-10297-9

Liu, J., Huang, Q., Xia, X., Shihab, E., Lo, D., & Li, S. (2020). Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks. *Proceedings 2020 ACM IEEE 42nd International Conference on Software Engineering Software Engineering in Society ICSE Seis 2020*, 1–10.

Melo, A., Fagundes, R., Lenarduzzi, V., & Santos, W. (2022). Identification and measurement of requirements technical debt in software development: A systematic literature review. *Journal of Systems and Software*, *194*. https://doi.org/10.1016/j.jss.2022.111483

Nabukenya, J. (2012). Combining case study, design science and action research methods for effective collaboration engineering research efforts, 343–352. https://doi.org/10.1109/HICSS.2012.162

Nayebi, M., Cai, Y., Kazman, R., Ruhe, G., Feng, Q., Carlson, C., & Chew, F. (2019). A longitudinal study of identifying and paying down architecture debt. *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 171–180. https://doi.org/10.1109/ICSE-SEIP.2019.00026

Rindell, K., Bernsmed, K., & Jaatun, M. G. (2019). Managing security in software: Or: How i learned to stop worrying and manage the security technical debt. *Proceedings of the 14th International Conference on Availability, Reliability and Security*. https://doi.org/10.1145/3339252.3340338

Rindell, K., & Holvitie, J. (2019). Security risk assessment and management as technical debt. *2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, 1–8. https://doi.org/10.1109/CyberSecPODS.2019.8885100

Rios, N., Mendes, L., Cerdeiral, C., Magalhães, A., Perez, B., Correal, D., Astudillo, H., Seaman, C., Izurieta, C., Santos, G., Santos, G., & Spínola, R. O. (2020). *Hearing the voice*

of software practitioners on causes, effects, and practices to deal with documentation debt (Vol. 12045 LNCS). https://doi.org/10.1007/978-3-030-44429-7_4

Rios, N., Freire, S., Pérez, B., Castellanos, C., Correal, D., Mendonça, M., Falessi, D., Izurieta, C., Seaman, C., & Spínola, R. (2021). On the relationship between technical debt management and process models. *IEEE Software*, *PP*. https://doi.org/10.1109/MS.2021.3058652

Rios, N., Mendonça, M., Seaman, C., & Spínola, R. (2019). Causes and effects of the presence of technical debt in agile software projects.

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Softw. Engg.*, *14*(2), 131–164. https://doi.org/10.1007/s10664-008-9102-8

Seyff, N., Betz, S., Groher, I., Stade, M., Chitchyan, R., Duboc, L., Penzenstadler, B., Venters, C., & Becker, C. (2018). Crowd-focused semi-automated requirements engineering for evolution towards sustainability. https://doi.org/10.1109/RE.2018.00-23

Siddiqi, M. A., Doerr, C., & Strydis, C. (2020). Imdfence: Architecting a secure protocol for implantable medical devices. *IEEE Access*, *8*, 147948–147964. https://doi.org/10.1109/ACCESS.2020.3015686

Snoeck, M., & Wautelet, Y. (2022). Agile merode: A model-driven software engineering method for user-centric and value-based development. *Software and Systems Modeling*, *21*, 1469–1494. https://doi.org/10.1007/s10270-022-01015-y

Venters, C., Capilla, R., Betz, S., Penzenstadler, B., Crick, T., Crouch, S., Nakagawa, E., Becker, C., & Carrillo, C. (2018). Software sustainability: Research and practice from a software architecture viewpoint. *Journal of Systems and Software*, *138*, 174–188. https://doi.org/10.1016/j.jss.2017.12.026

Venters, C., Capilla, R., Nakagawa, E., Betz, S., Penzenstadler, B., Crick, T., & Brooks, I. (2023). Sustainable software engineering: Reflections on advances in research and practice. *Information and Software Technology*, *164*. https://doi.org/10.1016/j.infsof.2023.107316

Villamizar, H., Kalinowski, M., Viana, M., & Fernández, D. M. (2018). A systematic mapping study on security in agile requirements engineering. https://doi.org/10.1109/SEAA.2018.00080

Vogel-Heuser, B., & Bi, F. (2021). Interdisciplinary effects of technical debt in companies with mechatronic products — a qualitative study. *Journal of Systems and Software*, *171*. https://doi.org/10.1016/j.jss.2020.110809

Vogelsang, A. (2020). Feature dependencies in automotive software systems: Extent, awareness, and refactoring. *Journal of Systems and Software*, *160*, 41–50. https://doi.org/10.1016/j.jss.2019.110458

Waltersdorfer, L., Rinker, F., Kathrein, L., & Biffl, S. (2020). Experiences with technical debt and management strategies in production systems engineering. *Proceedings 2020 IEEE*

ACM International Conference on Technical Debt Techdebt 2020, 41–50. https://doi.org/10.1145/3387906.3388627

Wattanakriengkrai, S., Maipradit, R., Hata, H., Choetkiertikul, M., Sunetnanta, T., & Matsumoto, K. (2018). Identifying design and requirement self-admitted technical debt using n-gram idf. *Proceedings 2018 9th International Workshop on Empirical Software Engineering in Practice Iwesep 2018*, 7–12. https://doi.org/10.1109/IWESEP.2018.00010

Wattanakriengkrai, S., Srisermphoak, N., Sintoplertchaikul, S., Choetkiertikul, M., Ragkhitwetsagul, C., Sunetnanta, T., Hata, H., & Matsumoto, K. (2019). Automatic classifying self-admitted technical debt using n-gram idf. *Proceedings Asia Pacific Software Engineering Conference APSEC*, *2019-Decem*, 316–322. https://doi.org/10.1109/APSEC48747.2019.00050

ID	Title	Authors	Year	Type	Source
S1	Actions and Impediments for Technical Debt Prevention: Results From a Global Family of Industrial Surveys	Freire et al.	2020	Conference Paper	Proceedings of the 35th Annual ACM Symposium on Applied Computing
S2	A Longitudinal Study of Identifying and Paying Down Architecture Debt	Nayebi et al.	2019	Conference Paper	ICSE-SEIP
S3	AGILE MERODE: A Model-Driven Software Engineering Method	Snoeck and Wautelet	2022	Journal Article	Software and Systems Modeling
S4	Automatic Classifying Self-Admitted Technical Debt Using N-Gram IDF	Wattanakriengkrai et al.	2019	Conference Paper	APSEC
S5	Causes and Effects of the Presence of Technical Debt in AGILE Projects	Rios et al.	2019	Conference Paper	SEAA
S6	Crowd-Focused Semi-Automated Requirements Engineering	Seyff et al.	2018	Conference Paper	IEEE RE
S7	Experiences With Technical Debt and With Its Management Strategies in Practice	Waltersdorfer et al.	2020	Conference Paper	TechDebt
S8	Feature Dependencies in Automotive Software Systems: A Multi-Method Study	Vogelsang	2020	Journal Article	Journal of Systems and Software

ID	Title	Authors	Year	Type	Source
S9	Fostering Continuous Value Proposition Innovation Through Freelancer Involvement: An Action Design Research Approach	Gupta et al.	2020	Journal Article	Sustainability
S10	Hearing the Voice of Software Practitioners on Documentation Debt: A Mapping Study	Rios et al.	2020	Book Chapter	LNCS
S11	Identification and Remediation of Self-Admitted Technical Debt in Issue Trackers	Li et al.	2020	Conference Paper	SEAA
S12	Identifying Design and Requirement Self-Admitted Technical Debt Using N-Gram IDF	Wattanakriengkrai et al.	2018	Conference Paper	IWESEP
S13	Identifying Self-Admitted Technical Debt Through Comment Analysis Using Contextual Vocabulary	Farias et al.	2020	Journal Article	Information and Software Technology
S14	IMDfence: Architecting a Secure Protocol for Implantable Medical Devices	Siddiqi et al.	2020	Journal Article	IEEE Access
S15	Interdisciplinary Effects of Technical Debt in Companies With Mechatronic Products	Vogel-Heuser and Bi	2021	Journal Article	Journal of Systems and Software

ID	Title	Authors	Year	Type	Source
S16	Is Using Deep Learning Frameworks Free? Measuring the Cost of DL Frameworks From a Technical Debt Perspective	Liu et al.	2020	Conference Paper	ICSE-SEIS
S17	Managing Security in Software: Or: How I Learned to Stop Worrying and Manage the Security Technical Debt	Rindell et al.	2019	Conference Paper	ARES
S18	On the Relationship Between Technical Debt Management and Process Models: A Case Study of a Start-Up Company	Rios et al.	2021	Journal Article	IEEE Software
S19	Requirements Quality is Quality in Use	Femmer and Vogelsang	2019	Journal Article	IEEE Software
S20	Security Risk Assessment and Management as Technical Debt	Rindell and Holvitie	2019	Conference Paper	Cyber Security Conference
S21	Software Sustainability: Research and Practice From a Software Architecture Viewpoint	Venters et al.	2018	Journal Article	Journal of Systems and Software
S22	Sustainable Software Engineering: Reflections on Advances in Research and Practice	Venters et al.	2023	Journal Article	Information and Software Technology
S23	Technical Debt Triage in Backlog Management: An Industrial Case Study	Besker et al.	2019	Conference Paper	TechDebt

ID	Title	Authors	Year	Type	Source
S24	The Evolution of Requirements Practices in Software Startups: A Grounded Theory of AGILE and Flexible Development	Gralha et al.	2018	Conference Paper	ICSE
S25	Towards a Holistic Definition of Requirements Debt	Lenarduzzi and Fucci	2019	Conference Paper	ESEM
S26	Towards Optimal Quality Requirements Documentation in AGILE Software Development	Behutiye et al.	2022	Journal Article	Journal of Systems and Software
S27	A Systematic Mapping Study on Security in AGILE Requirements Engineering	Villamizar et al.	2018	Conference Paper	SEAA
S28	Automatic Identification of Self-Admitted Technical Debt From Four Different Sources	Li et al.	2023	Journal Article	Empirical Software Engineering
S29	Identification and Measurement of Requirements Technical Debt: A Case Study in a Large-Scale AGILE Environment	Melo et al.	2022	Journal Article	Journal of Systems and Software
S30	Is Using Deep Learning Frameworks Free? Characterizing Technical Debt in Deep Learning Frameworks	Liu et al.	2020	Conference Paper	ICSE-SEIS